

Month 4 CRM Software Project

Client Overview:

The client requires a full-stack **web-based CRM system** to manage:

- Leads
- Customers
- Tasks
- Sales Pipelines

 **Goal:** Improve team productivity and customer engagement through centralized CRM.

Project Objective:

Develop a **secure, scalable CRM** using:

- **Backend:** Spring Boot (Java)
 - **Frontend:** React.js
 - **Database:** MySQL
 - **Authentication:** JWT + Spring Security
 - **Authorization:** Role-based (Admin & Sales Rep)
-

Functional Modules

User Management & Authentication

➤ API Endpoints

- POST /api/register
- POST /api/login
- GET /api/users/me

➤ Data Fields

- Full Name, Email, Password (Encrypted), Role (Admin/Sales), CreatedAt

➤ Security

- JWT-based login
- Role-based access using **Spring Security**
- Token stored in **localStorage** (Frontend)

- Protect routes on frontend via **React Router**
-

2 Customer Management

► **API Endpoints**

- GET /api/customers
- GET /api/customers/{id}
- POST /api/customers
- PUT /api/customers/{id}
- DELETE /api/customers/{id}

► **Data Fields**

- Name, Email, Phone, Company, Address, Assigned Sales Rep, Notes
-

3 Lead Management

► **API Endpoints**

- GET /api/leads
- POST /api/leads
- PUT /api/leads/{id}
- DELETE /api/leads/{id}

► **Data Fields**

- Name, Contact Info, Source (Referral, Ads, Web), Status, Assigned Rep
-

4 Task Management

► **API Endpoints**

- GET /api/tasks
- POST /api/tasks
- PUT /api/tasks/{id}
- DELETE /api/tasks/{id}

► **Data Fields**

- Title, Description, Due Date, Priority, Assigned To, Status
-

Sales Management

► API Endpoints

- GET /api/sales
- POST /api/sales
- GET /api/sales/{id}
- PUT /api/sales/{id}

► Data Fields

- Customer ID, Amount, Status, Date, Assigned Rep
-

Database Schema (MySQL)

Tables:

- users (Admin / Sales)
- customers
- leads
- tasks
- sales

Relationships:

- One user (Sales Rep) → many leads, customers, tasks, sales
 - One customer → many sales
-

Security Implementation

Area	Tech Used
------	-----------

Token	JWT
-------	-----

Authorization	Spring Security
---------------	-----------------

Frontend Protection React Router + Token Validation

API Documentation

- Use **Swagger / OpenAPI**
 - Access at: <http://localhost:8080/swagger-ui.html>
-

Frontend (React.js)

Key Pages:

- Login / Register
- Dashboard (Overview of sales, leads, tasks)
- Customers
- Leads (Filter + Assign)
- Tasks (My tasks, assign, mark done)
- Sales (Deals, status updates)

Role-based UI:

- Admin: Full access
 - Sales Rep: Limited view/edit access
-

Project Structure

Backend (Spring Boot)

crm-backend/

├─ controller/

├─ service/

├─ repository/

├─ model/

├─ security/

├─ dto/

└─ application.properties

Frontend (React)

crm-frontend/

├─ src/

| └─ components/

| └─ pages/

| └─ services/ ← Axios API calls

| └─ utils/

| └─ App.js

4-Week Project Timeline

Week	Backend	Frontend
1	User Auth (JWT + Spring Security) DB Schema Setup	Login & Register pages
2	CRUD for Customers & Leads	Customers & Leads Page
3	Task Module + Sales Module	Task & Sales Pages
4	Swagger Docs Pagination Testing	Dashboard UI Final UX polish Role-based rendering

Deliverables

- Complete Frontend & Backend Code
 - Postman Collection
 - Swagger API Docs
 - Fully Running Application
 - Optional: GitHub Repo
-

Testing Strategy

Area	Tools
Backend Unit Tests	JUnit + Mockito
API Testing	Postman, Swagger
Frontend	Manual testing (Optional: Jest)

Optional Enhancements

-  Notification system for tasks
-  Email reminders (follow-ups)
-  Export reports (PDF, CSV)
-  Dark mode
-  Dashboard charts using Chart.js or ApexCharts

NOTE: You're free to build the backend in your own way, using either **Java (Spring Boot)** or **Node.js with MongoDB**—whichever you prefer or feel more comfortable with for the CRM Software project.